Лабораторна робота №4

«Власні віджети та компоненти інтерфейсу»

Мета

Ознайомитися з підходом до створення власних віджетів у Flutter та навчитися виділяти повторювані елементи інтерфейсу (заголовки, інформаційні блоки, кнопки, картки) в окремі компоненти. Закріпити навички роботи з layout-віджетами (Row, Column, Container, Padding, Expanded, Flexible, Spacer) та навчитися передавати дані через конструктор віджета.

Теоретичні відомості

1. Навіщо створювати власні віджети?

Власні віджети (компоненти) дозволяють:

- зменшити дублювання коду;
- зробити структуру інтерфейсу більш зрозумілою;
- спростити підтримку та зміну дизайну;
- повторно використовувати однакові елементи (картки, кнопки, блоки);
- логічно розділити екран на незалежні частини.

У Flutter будь-який елемент інтерфейсу може бути винесений в окремий віджет.

2. StatelessWidget — основа власних компонентів інтерфейсу

StatelessWidget — це базовий тип віджета у Flutter, який використовується для відображення інтерфейсу без збереження внутрішнього стану.

Основні характеристики StatelessWidget:

- не має змінного стану;
- не змінюється після побудови;
- повністю залежить від даних, переданих через конструктор;
- при зміні даних віджет перебудовується зовні, а не керує станом самостійно.

StatelessWidget підходить для створення:

- заголовків;
- карток;
- інформаційних блоків;
- кнопок;
- декоративних елементів;
- будь-яких повторюваних UI-компонентів.

StatelessWidget доцільно використовувати, якщо:

- віджет лише відображає дані;
- віджет не реагує на події зміною власного стану;
- усі дані надходять через параметри конструктора.

Приклад простого StatelessWidget

```
class TitleBlock extends StatelessWidget {
 final String text;

 const TitleBlock({
   super.key,
   required this.text,
 });

 @override
 Widget build(BuildContext context) {
   return Text(
     text,
     style: const TextStyle(
       fontSize: 22,
       fontWeight: FontWeight.bold,
     ),
   );
 }
}
```

3. Передача даних у власні віджети

Дані у власні віджети передаються через конструктор:

- текст (String);
- іконки (IconData);
- кольори (Color);
- обробники подій (VoidCallback).

Використання required робить параметри обов’язковими.

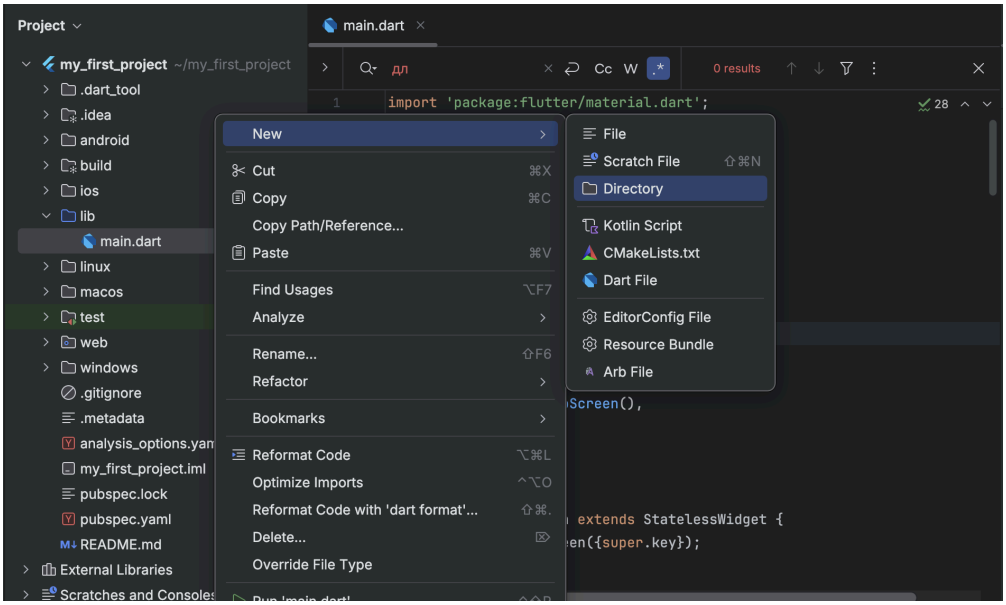
4. Створення структури папок проєкту

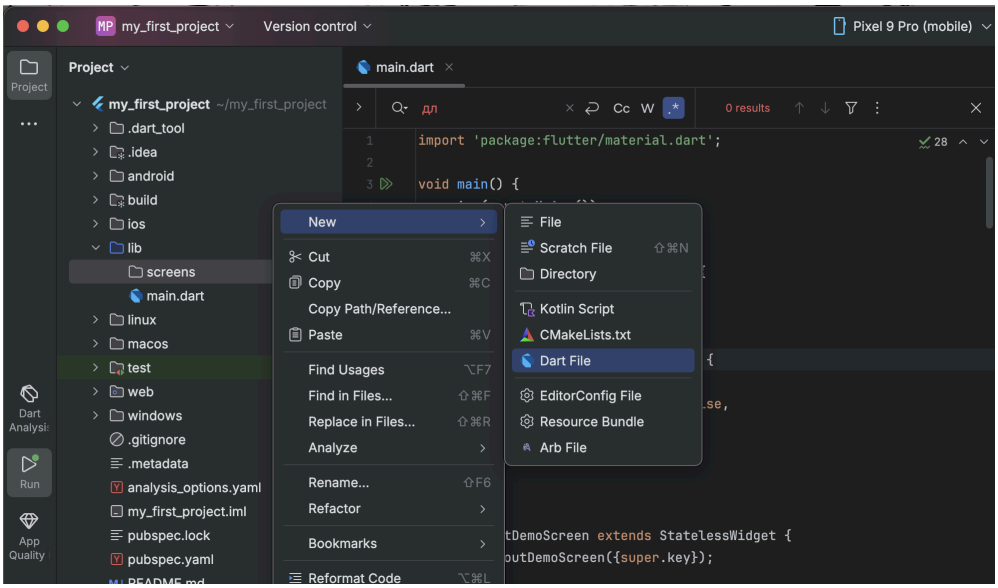
Для лабораторної роботи №4 не рекомендується розміщувати всі віджети у файлі main.dart.

Прийнято розділяти екрани та власні компоненти по окремих папках.

Приклад структури проєкту

```
lib/
  main.dart
  screens/
    lab4_screen.dart
  widgets/
    header_block.dart
    info_card.dart
    action_buttons_row.dart
    bottom_image.dart
```





Призначення файлів:

- main.dart — точка входу в застосунок;
- lab4_screen.dart — основний екран лабораторної;
- widgets/ — власні UI-компоненти.

5. Імпорт власних віджетів у Flutter

Для використання стандартних віджетів Flutter у файлі необхідно підключити бібліотеку:

```
import 'package:flutter/material.dart';
```

Цей імпорт надає доступ до базових UI-компонентів Flutter (Scaffold, AppBar, Text, Row, Column, Container, кнопки тощо).

Після винесення власних компонентів у окремі файли, їх необхідно імпортувати у файл екрана, де вони використовуються.

Приклад імпорту власних віджетів:

```
import 'package:flutter/material.dart';
import '../widgets/header_block.dart';
import '../widgets/info_card.dart';
import '../widgets/action_buttons_row.dart';
import '../widgets/bottom_image.dart';
```

У цьому прикладі:

- ../widgets/ — шлях до папки з власними компонентами;
- кожен файл містить окремий StatelessWidget;
- імпорт дозволяє повторно використовувати створені віджети на різних екранах.

Рекомендація: кожен власний віджет доцільно розміщувати в окремому файлі з назвою, що відповідає назві класу (header_block.dart → HeaderBlock).

```dart
import 'package:flutter/material.dart';

class HeaderBlock extends StatelessWidget {
 final String title;
 final String description;

 const HeaderBlock({
   super.key,
   required this.title,
   required this.description,
 });

 @override
 Widget build(BuildContext context) {
   return Container(
     padding: const EdgeInsets.all(16),
     decoration: BoxDecoration(
       color: Colors.blueAccent,
       borderRadius: BorderRadius.circular(12),
     ),
     child: Column(
       crossAxisAlignment: CrossAxisAlignment.start,
       children: [
         Text(
           title,
           style: const TextStyle(
             color: Colors.white,
             fontSize: 22,
             fontWeight: FontWeight.bold,
           ),
         ),
         const SizedBox(height: 8),
         Text(
           description,
```

```
                style: const TextStyle(
                  color: Colors.white70,
                  fontSize: 16,
                ),
              ),
            ],
          ),
        );
      }
    }
```

Так буде виглядати .main

```
import 'package:flutter/material.dart';
import 'screens/lab4_screen.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return const MaterialApp(
      debugShowCheckedModeBanner: false,
      home: Lab4Screen(),
    );
  }
}
```

Приклад файлу lab4_screen.dart, коли зробили винесли код у власні віджети:

```
import 'package:flutter/material.dart';
import '../widgets/header_block.dart';
import '../widgets/info_card.dart';
import '../widgets/action_buttons_row.dart';
import '../widgets/bottom_image.dart';
```

```dart
class Lab4Screen extends StatelessWidget {
 const Lab4Screen({super.key});

 @override
 Widget build(BuildContext context) {
   return Scaffold(
     appBar: AppBar(title: const Text('Лабораторна №4')),
     body: Padding(
       padding: const EdgeInsets.all(16),
       child: Column(
         crossAxisAlignment: CrossAxisAlignment.stretch,
         children: [
           const HeaderBlock(
             title: 'Назва екрана',
             description: 'Приклад використання власних віджетів',
           ),

           const SizedBox(height: 12),

           const InfoCard(
             icon: Icons.info_outline,
             title: 'Інформаційний блок',
             subtitle: 'Цей блок винесено в окремий компонент',
           ),

           const SizedBox(height: 12),

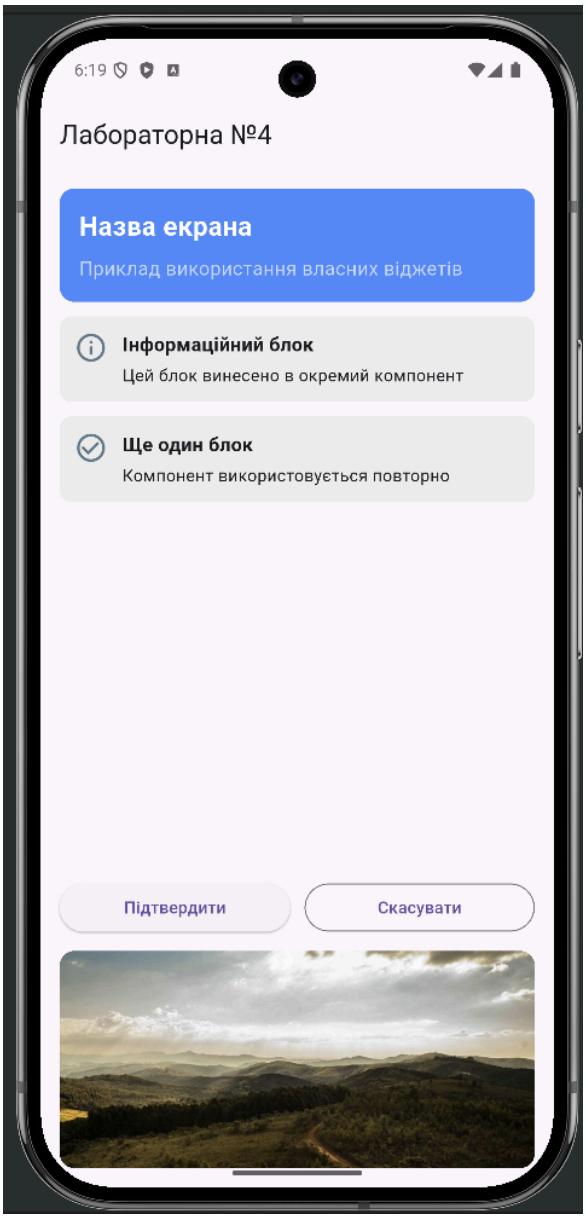
           const InfoCard(
             icon: Icons.check_circle_outline,
             title: 'Ще один блок',
             subtitle: 'Компонент використовується повторно',
           ),

           const Spacer(),

           ActionButtonsRow(
             primaryText: 'Підтвердити',
             secondaryText: 'Скасувати',
             onPrimary: () {},
             onSecondary: () {},
           ),
```

```
            const SizedBox(height: 12),

            const BottomImage(
              imageUrl: 'https://picsum.photos/800/400',
            ),
          ],
        ),
      ),
    );
  }
}
```



Завдання на лабораторну роботу

1. Продовжити роботу над проєктом, розпочатим у Лабораторній роботі №3.
2. На основі готового макета з Лабораторної роботи №3 переробити екран, використовуючи власні віджети.
3. Виділити елементи інтерфейсу в окремі компоненти (StatelessWidget), зокрема:
    - заголовок або верхній інформаційний блок;
    - інформаційну картку або текстовий блок;
    - блок кнопок дії;
    - блок із зображенням або іконкою.
4. Для кожного власного віджета:
    - створити окремий файл у папці widgets/;
    - реалізувати передачу даних через конструктор (required параметри);
5. Організувати структуру проєкту з використанням папок:
    - screens/ — для екранів;
    - widgets/ — для власних компонентів.
6. У файлі екрана (lab4_screen.dart) імпортувати та використати створені власні віджети.
7. Під час побудови інтерфейсу використати layout-віджети:
    - Column, Row;
    - Container, Padding;
    - Expanded або Flexible;
    - Spacer.
8. Забезпечити:
    - логічне розміщення елементів;

- наявність відступів між блоками;
- відсутність помилок верстки (overflow).

9. Перевірити коректну роботу застосунку на емуляторі або фізичному пристрої.

Контрольні запитання

1. Що таке власний віджет у Flutter?
2. Для чого створюють власні компоненти інтерфейсу?
3. Чому в цій лабораторній роботі використовуються саме StatelessWidget?
4. Як передати дані у власний віджет?
5. Для чого використовується ключове слово required у конструкторі?
6. Чому не рекомендується розміщувати весь код у файлі main.dart?
7. Яке призначення папок screens та widgets у Flutter-проєкті?
8. Які переваги має повторне використання віджетів у Flutter?